Digital Design Lab

Ratko Pilipović

2026-10-12

Table of contents

Preface **1**

I. Lab Exercises **3**

1. First exercise: Tools and the Digital Design Cycle **5**
1.1. The digital design cycle 5
1.2. Tools used in this course 6
1.3. Installing Anvil 8
1.4. Your first design: basic_comb 9
1.5. Starter files 13
1.6. Assignment 13

Preface

Every processor, controller, and screen you have ever used began as a handful of logic gates someone arranged on purpose. This book is about learning to do that arranging yourself — not by reading about it, but by designing, simulating, and running your own digital circuits on real hardware.

Over nine labs, you will go from your first combinational circuit to a complete FPGA-based System on Chip: a HIPV, a Risc-V based processor, talking to peripherals you wrote, over a bus you configured, validated with a real oscilloscope. Along the way you will hit the same problems every digital designer hits — a signal that glitches, a counter that free-runs, a testbench that lies to you — and learn to reason your way out of them.

That debugging instinct, more than any single lab, is the point.

Note to Students: These notes are intended solely for educational purposes. They are not a replacement for attending lectures or consulting textbooks, but rather a complementary resource to support your learning.

Part I.

Lab Exercises

1. First exercise: Tools and the Digital Design Cycle

This first lab has two goals. First, to introduce the **digital design cycle** — the sequence of steps every digital circuit goes through, from an idea to a working bitstream on real hardware — so that you understand *why* each later lab does what it does. Second, to get your machine set up with the toolchain we use throughout the course, so the remaining labs are about designing hardware rather than fighting installers.

1.1. The digital design cycle

Every design in this course, however small, passes through the same stages:

1. **Specification** — describe, in plain language, what the circuit must do: its inputs, its outputs, and the relationship between them.
2. **RTL design entry** — write that behavior in a hardware description language. In this course, that language is SystemVerilog.
3. **Functional verification (simulation)** — before touching real hardware, simulate the design against a testbench to check it behaves as specified. Bugs are far cheaper to find here than on a board.
4. **Synthesis** — translate the RTL description into a gate-level netlist: a graph of primitive logic cells and the wires connecting them.

5. **Place & route (implementation)** — map that netlist onto the FPGA's actual resources (logic cells, routing wires, I/O pins), respecting the timing and pin constraints you provide.
6. **Bitstream generation** — package the placed-and-routed design into the binary configuration file the FPGA fabric loads on power-up.
7. **Programming** — load the bitstream onto the physical board.
8. **Hardware validation** — check that the programmed board actually does what you specified in step 1. If it doesn't, you go back to step 1 or 2 and iterate.

Steps 3–7 are exactly what a toolchain automates for you. Knowing this sequence matters because when something goes wrong, the first useful question is *which stage failed* — a syntax error is a design-entry problem, a mismatch with the testbench is a verification problem, an unroutable design is a place & route problem, and an LED that doesn't light despite a clean build is a hardware/pin-constraint problem. Each points you to a different fix.

1.2. Tools used in this course

In previous years this course used **Xilinx Vivado**, a proprietary, vendor-specific design suite. **Starting this year, we use Anvil** — a small, open-source CLI that drives an entirely open FPGA flow (no license, no account, no multi-GB installer). Anvil orchestrates the same stages described above using open tools:

Design cycle stage	Tool
Design entry	SystemVerilog (you write `top.sv`)
Simulation	Icarus Verilog (`iverilog`) + GTKWave, via `anvil test`

Design cycle stage	Tool
Synthesis	`sv2v` (SystemVerilog → Verilog) feeding **Yosys**
Place & route	**VPR** (part of the F4PGA project — Yosys + VPR together)
Bitstream generation	F4PGA
Programming	`openFPGALoader`

Anvil itself just wires these tools together behind a handful of commands (`anvil init`, `anvil build`, `anvil test`, `anvil program`) so you don't invoke each one by hand.

Anvil supports two FPGA boards, the **Digilent Nexys A7 100T** and the **Nexys A7 50T**. Both boards are used across the exercises in this course.

> **i** Nexys A7 50T
>
> Anvil supports both the **Nexys A7 100T** and the **Nexys A7 50T** (support for the 50T was added in Anvil 1.1.0).
> **Your own computer:** a normal installation works out of the box. If you installed Anvil before the 50T was added, run `anvil update` once.
> **Classroom computers:** these have an older Anvil and you cannot run `anvil update` there (it needs `sudo`). Apply the classroom patch instead; it needs no administrator rights and changes only files in your home directory:
>
> ```
> wget https://gist.githubusercontent.com/RatkoFri/d12d34610a58260a58a803eec74e7185/raw/anvi
> bash anvil-nexys50t.sh
> source ~/.bashrc
> ```
>
> Run it once per account; running it again does no harm. Check the

result with `anvil boards` (both boards should be listed) and `anvil doctor`. Then start a 50T project with:

```
anvil init --board Nexys-A7-50T --example switches-leds
```

1.3. Installing Anvil

Prerequisites: a native **Ubuntu 22.04 LTS** machine, or **WSL2** on Windows (the docs call out the handful of WSL-only steps, mainly around forwarding the board's USB connection, as "WSL only"). Docker support is not available yet.

The recommended path is the automated setup script:

```
curl -fsSL https://raw.githubusercontent.com/LogiSmith/toolchain-setup/main/i
```

If you'd rather review the script before running it (always a reasonable instinct when piping a script into a shell):

```
git clone https://github.com/LogiSmith/toolchain-setup.git
cd toolchain-setup
less install.sh
./install.sh
```

A successful run ends with a green `Toolchain installed and all tests passed` banner. Open a **new** shell afterward (so your updated `PATH`/aliases take effect) and verify with:

```
anvil doctor
```

For manual, tool-by-tool installation steps, WSL2-specific USB forwarding, and troubleshooting, see the full LogiSmith installation docs — that page is the canonical reference and is more likely to stay current than a copy pasted here.

1.4. Your first design: basic_comb

With `anvil doctor` passing, build a first design that exercises switches and LEDs directly, with no clock involved: a small combinational circuit.

```
mkdir basic_comb && cd basic_comb
anvil init --board Nexys-A7-100T
```

This scaffolds a project:

```
basic_comb/
   config.json    # board + module configuration
   top.sv         # top-level design - edit this
   *.xdc          # pin constraints, copied from the board master
   tb/            # testbenches
   Makefile
```

1.4.0.1. What's in `config.json`

`config.json` is the per-project manifest every `anvil` command reads (`build`, `test`, `program`). `anvil init --board Nexys-A7-100T` fills it in for you, so you won't normally hand-edit it in this lab — but it's worth knowing what's there:

```
{
  "project": "basic_comb",
  "board": "Nexys-A7-100T",
  "top": "top",
  "target": "nexys4ddr",
  "partname": "xc7a100tcsg324-1",
  "device": "artix7",
  "ofl_board": "nexys_a7_100",
  "xdc": "Nexys-A7-100T-Master.xdc"
}
```

- **project** — the project's name, defaulting to the directory name.
- **board** — which entry of Anvil's board registry (**boards.json**) this project targets; everything below **top** is copied in from that registry at **anvil init** time.
- **top** — the top-level module name Anvil looks for when it builds your design. It defaults to **top**, which is exactly why the module in **top.sv** needs to be named **top** rather than **basic_comb**.
- **partname**, **device**, **target**, **ofl_board**, **xdc** — board-specific details (exact FPGA part, device family, Makefile build target, **openFPGALoader** board identifier, and master constraints filename) that synthesis, place & route, and programming each need. You won't touch these directly for the Nexys A7 100T.
- **modules** (not shown above, since this design has none) — a list of reusable RTL module dependencies with pinned versions, e.g. **"uart@1.0.0"**, added automatically by **anvil addmodule**. You'll meet this once a later lab pulls in a shared peripheral block.

Replace the contents of **top.sv** with three switch inputs feeding four LED outputs through simple boolean logic. Anvil expects the top-level module in **top.sv** to be named **top**, so the logic below is the same as the basic_comb.sv starter file, just renamed from **basic_comb** to **top**:

```
// basic_comb - simple 3-input combinational logic
module top (
    input  logic a,        // SW0
    input  logic b,        // SW1
    input  logic c,        // SW2
    output logic [3:0] y   // LED0-LED3
);
    assign y[0] = a & b | c;
    assign y[1] = a & c;
    assign y[2] = a | c;
    assign y[3] = a & b & c;
endmodule
```

1.4.0.2. Explanation

- **Continuous assignment**: each `assign` statement drives its output from a boolean expression of the inputs, re-evaluated any time `a`, `b`, or `c` changes. There is no clock and no memory — the outputs are a pure function of the current switch positions, which is exactly what "combinational logic" means.
- **This is the design-entry stage**: `assign` is the idiomatic way to describe combinational logic in SystemVerilog, the same way `always_ff` (which you'll meet in the next lab) is the idiom for sequential logic.
- **Four independent outputs**: `y[0]`–`y[3]` are four separate circuits sharing the same three inputs — nothing requires a module to have only one output bit.

Uncomment (or check) the pins for `a`, `b`, `c`, and `y` in the generated `.xdc` file — they should look like this:

```
set_property PACKAGE_PIN J15 [get_ports {a}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {a}]

set_property PACKAGE_PIN L16 [get_ports {b}]
set_property IOSTANDARD LVCMOS33 [get_ports {b}]

set_property PACKAGE_PIN M13 [get_ports {c}]
set_property IOSTANDARD LVCMOS33 [get_ports {c}]

set_property PACKAGE_PIN H17 [get_ports {y[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {y[0]}]

set_property PACKAGE_PIN K15 [get_ports {y[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {y[1]}]

set_property PACKAGE_PIN J13 [get_ports {y[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {y[2]}]

set_property PACKAGE_PIN N14 [get_ports {y[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {y[3]}]
```

This maps `a`, `b`, `c` to switches SW0–SW2 and `y[0]`–`y[3]` to LED0–LED3 on the Nexys A7 100T. Unlike Blinky, there's no `create_clock` line here — a purely combinational design has no clock port for the timing analyzer to check.

Then build and program the board:

```
anvil build       # sv2v → Yosys → VPR → bitstream, a few seconds
anvil program     # flash the bitstream to the board via openFPGALoader
```

If everything worked, flipping SW0–SW2 should change the LED0–LED3 pattern according to the four equations above.

1.5. Starter files

- basic_comb.sv — the same design used in the walkthrough above, as a ready-to-copy file (module named `basic_comb`; rename to `top` when you copy it into `top.sv`, as noted above).

1.6. Assignment

1. Run `anvil doctor` and confirm every check passes. If something fails, consult the troubleshooting guide before asking for help.
2. Build and program the `basic_comb` design above, and confirm the LED pattern matches the equations for every switch combination.
3. Extend the design with a fifth output, `y[4] = a ^ b ^ c` (XOR of all three switches), wired to LED4. Rebuild, reprogram, and confirm the new output on hardware.